

Aula 1 - Introdução

Prof. Danilo de Quadros Maia Filho

Departamento:
Engenharia de Software

1 Introdução ao Curso

- Sobre a Disciplina
- Medição
- Experimentação
- Cuidado com as Métricas
- Métricas Modernas em Engenharia de Software
- Medição na Era da IA Generativa
- Sobre o Professor
- Planejamento da Disciplina
- Regras e Combinados

2 Primeiras Definições

1 Introdução ao Curso

- Sobre a Disciplina
- Medição
- Experimentação
- Cuidado com as Métricas
- Métricas Modernas em Engenharia de Software
- Medição na Era da IA Generativa
- Sobre o Professor
- Planejamento da Disciplina
- Regras e Combinados

2 Primeiras Definições

Sobre a Disciplina

A disciplina de Medição e Experimentação em Engenharia de Software é fundamental em um curso de graduação porque forma a base para decisões técnicas baseadas em dados, promove a qualidade do software e prepara os estudantes para um ambiente profissional onde evidências superam opiniões.

O processo de medição está presente diariamente em tudo o que fazemos. Exemplos: Culinária, sono, academia, comunicação social, relacionamentos, sede, fome, uso de celular, relógio, computadores, ar condicionado, direção de carros e motos, cortes de cabelo.

Hoje, boa parte dessa medição é feita por dispositivos que carregamos o tempo todo: Apple Watch, Whoop, Oura Ring, Strava, contadores de passos e apps de sono. A esse hábito de medir constantemente a própria vida com dados, dá-se o nome de **Quantified Self**.

O Movimento Quantified Self

O termo **Quantified Self** foi cunhado em 2007 por Gary Wolf e Kevin Kelly, editores da revista *Wired*, para descrever pessoas que usam tecnologia para coletar dados sobre seus próprios hábitos e desempenho.

Exemplos atuais:

- **Apple Watch / Whoop / Oura Ring:** sono, frequência cardíaca, recuperação
- **Strava:** desempenho em corridas e pedaladas
- **Apps de sono:** ciclos de sono profundo/leve
- **Apps financeiros:** gastos categorizados automaticamente

A lógica é a mesma que usaremos em engenharia de software: **sem medir, não sabemos se estamos melhorando.**

Sem medir, não sabemos quão bom ou ruim é um processo, produto ou equipe.

Exemplo:

Uma empresa decidiu trocar sua arquitetura monolítica por microsserviços. Sem medições, não perceberam que a complexidade aumentou e o tempo de entrega triplicou. Medir métricas como lead time, número de bugs ou uso de CPU teria evitado o erro.

A medição permite aplicar melhoria contínua usando ciclos como PDCA (Plan–Do–Check–Act).

Exemplo:

A Microsoft mede tempo médio de resolução de bugs por equipe e, com isso, promove ajustes de práticas ágeis que aumentam eficiência e qualidade.

Grandes empresas como Google, Meta, Microsoft e iFood usam experimentos A/B, métricas de produtividade e desempenho como parte da cultura de engenharia.

Exemplo:

O Google usa métricas como MTTR (Mean Time to Repair) e mudança de código por dev por semana para otimizar times.

<https://www.youtube.com/@DavesGarage>



Dave's Garage ✓

@DavesGarage · 1,02 mi de inscritos · 499 vídeos

Windows History, Windows vs Linux Comparisons, Arduino Project Tutorials, Shop Projects...mais

[youtube.com/@davepl](https://www.youtube.com/@davepl) e mais 5 links



Inscrito ▾

Figura 3: DavesGarage



Inside Task Manager with Dave Plummer, the Original Author



Dave's Garage ✓

1,02 mi de inscritos



Inscrito ▼



127 mil



Compartilhar



Figura 4: Task Manager

Outras Fontes para Acompanhar Métricas de Engenharia

Além de canais como o DavesGarage, algumas fontes são referências centrais na indústria para acompanhar o estado da arte em métricas de engenharia de software:

- **DORA State of DevOps Report** (anual, desde 2014) — dora.dev
- **GitHub Octoverse** (anual) — octoverse.github.com
- **Stack Overflow Developer Survey** (anual) — maior pesquisa da comunidade dev do mundo

Vamos usar o **DORA Report** como base para uma das métricas mais modernas de desempenho em engenharia de software, que veremos mais à frente nesta aula.

Medições objetivas permitem apresentar resultados com clareza para clientes, gerentes e equipes técnicas.

Experimentação em software permite testar hipóteses com segurança, como:

- 1 “Essa nova interface reduz erros do usuário?”
- 2 “Este novo algoritmo consome menos memória?”

Exemplo:

A Amazon testa alterações de interface via experimentos A/B, medindo conversões em tempo real. Algumas mudanças simples, como mudar a cor de um botão, aumentaram vendas em milhões.

Nem Toda Métrica é Uma Boa Métrica

Até agora vimos exemplos onde medir trouxe benefícios claros. Mas medir mal — ou medir a coisa errada — pode ser pior do que não medir.

Vamos ver alguns princípios clássicos que todo profissional que trabalha com métricas precisa conhecer.

Lei de Goodhart

"Quando uma medida se torna um alvo, ela deixa de ser uma boa medida."

Popularizada pela antropóloga Marilyn Strathern (1997), a partir de uma ideia do economista Charles Goodhart (1975).

Exemplo em software:

Uma empresa define "linhas de código por dia" como métrica de produtividade. Os desenvolvedores passam a escrever código mais verboso e repetitivo — a métrica sobe, mas o valor entregue não aumenta (ou até piora).

O Efeito Cobra

A lenda do Efeito Cobra

Na Índia colonial britânica, o governo pagava uma recompensa por cada pele de cobra entregue, para reduzir a população de cobras venenosas em Délhi.

Resultado: a população local passou a **criar cobras** só para vendê-las e ganhar a recompensa. Quando o governo percebeu e cancelou o pagamento, os criadores soltaram as cobras — e a população de cobras selvagens **aumentou**.

Mesmo princípio da Lei de Goodhart: otimizar a métrica, não o objetivo real.

A Falácia de McNamara

Robert McNamara foi Secretário de Defesa dos EUA durante a Guerra do Vietnã. Sua equipe usava a **contagem de corpos inimigos** (*body count*) como principal métrica de sucesso militar.

Os 4 passos da Falácia (Charles Handy):

- 1 Medir o que é facilmente mensurável
- 2 Desconsiderar o que não pode ser medido facilmente, ou atribuir-lhe um valor quantitativo arbitrário
- 3 Presumir que o que não pode ser medido facilmente não é importante
- 4 Dizer que o que não pode ser medido facilmente, na verdade, não existe

Estudo de Caso: Wells Fargo (2016)

- O banco definiu metas agressivas de **venda cruzada** (cross-selling) por funcionário.
- Para bater as metas, funcionários abriram **milhões de contas e cartões de crédito falsos**, sem consentimento dos clientes.
- A métrica de "produtos vendidos por cliente" subiu — o valor real para o cliente e para o banco, não.

Consequência: multa de US\$ 3 bilhões, demissão do CEO, dano permanente à reputação.

Estudo de Caso: Volkswagen — Dieselgate (2015)

- A VW instalou um **software** nos motores a diesel capaz de **detectar** quando o carro estava em um teste oficial de emissões.
- Durante o teste, o software reduzia artificialmente a emissão de poluentes — fora dele, a emissão real chegava a ser dezenas de vezes maior que o permitido.
- A métrica de "conformidade no teste" foi otimizada diretamente — não a poluição real.

Consequência: dezenas de bilhões de dólares em multas e acordos, executivos processados criminalmente.

Então, Como Medir Bem?

As histórias anteriores têm um padrão em comum: uma métrica **única**, **fácil de otimizar** e **fácil de enganar**.

A resposta da engenharia de software moderna não é abandonar as métricas — é usar **conjuntos de métricas multidimensionais**, difíceis de "jogar" isoladamente.

Vamos ver dois frameworks que fazem exatamente isso.

Criadas pela pesquisa **DORA** (*DevOps Research and Assessment*), hoje parte do Google, e popularizadas pelo livro *Accelerate* (Forsgren, Humble & Kim, 2018).

As 4 métricas-chave:

- **Deployment Frequency** — com que frequência a equipe entrega em produção
- **Lead Time for Changes** — tempo entre um commit e ele estar em produção
- **Change Failure Rate** — % de mudanças que causam falha em produção
- **Mean Time to Restore (MTTR)** — tempo médio para recuperar de uma falha

Juntas, classificam times em **Elite / High / Medium / Low performers**.

Por que 4 Métricas, e Não 1?

Repare: as 4 métricas DORA se **balanceiam** entre si.

- Otimizar só **velocidade** (deploy frequency, lead time) sem cuidado gera mais falhas (change failure rate sobe).
- Otimizar só **estabilidade** (nunca falhar) tende a travar a velocidade de entrega.

Isso é uma defesa direta contra a **Lei de Goodhart**: um time não consegue "jogar" as 4 métricas ao mesmo tempo sem realmente melhorar o processo.

Proposto por pesquisadores da Microsoft e GitHub (Forsgren, Storey, Nagappan et al., 2021) como resposta direta à pergunta: *"Como medir produtividade de um desenvolvedor sem cair em armadilhas como linhas de código ou nº de commits?"*

As 5 dimensões:

- **S**atisfaction and well-being — satisfação e bem-estar
- **P**erformance — resultado do trabalho, não apenas volume
- **A**ctivity — ações realizadas (commits, PRs, deploys)
- **C**ommunication and collaboration — qualidade da colaboração
- **E**fficiency and flow — fluxo sem interrupções desnecessárias

Produtividade de desenvolvedor **não é um número único**.

Nenhuma dimensão isolada do SPACE deve ser usada como meta — exatamente para evitar o problema que vimos com Goodhart, McNamara, Wells Fargo e Volkswagen.

Regra prática: sempre que alguém propuser avaliar produtividade com **1 métrica só**, desconfie.

O Novo Desafio: Medir Produtividade com IA

Ferramentas como **GitHub Copilot**, **ChatGPT** e **Claude** mudaram como código é escrito — parte relevante do código hoje é **sugerida por IA**, não digitada do zero.

Isso quebra ainda mais métricas ingênuas:

- "Linhas de código por dia": IA gera linhas em segundos, sem relação com esforço ou qualidade.
- "Nº de commits": pode não refletir a dificuldade real da tarefa.

A Lei de Goodhart fica **ainda mais perigosa** na era da IA generativa.

O Que a Pesquisa Mostra

GitHub (2022)

Estudo controlado com desenvolvedores usando Copilot completou uma tarefa de programação cerca de **55% mais rápido** que o grupo sem a ferramenta.

DORA State of DevOps Report (2024)

Incluiu, pela primeira vez, perguntas sobre adoção de IA. Achado principal: adoção de IA tem efeito **positivo, porém pequeno** na produtividade individual — e pode até **piorar** a estabilidade de entrega se os processos de revisão de código não forem adaptados.

Ou seja: "eu codifico mais rápido" $\neq$ "a organização entrega mais valor".

Controvérsia: A Proposta da McKinsey (2023)

Em 2023, a consultoria **McKinsey** publicou um framework para medir "produtividade de desenvolvedor", incluindo métricas como % de código gerado por IA e contagem de PRs.

A proposta gerou forte **reação negativa** da comunidade — incluindo uma crítica pública conjunta de **Kent Beck** (criador do Extreme Programming) e **Gergely Orosz** (autor do The Pragmatic Engineer), amplamente compartilhada.

Crítica central: o framework reduzia trabalho de engenharia complexo a métricas individuais simplistas e facilmente 'jogáveis' - o mesmo erro de sempre.

Pergunta para reflexão:

Se o seu time usa Copilot ou ChatGPT no dia a dia, qual seria uma métrica capaz de capturar **valor real** entregue — sem cair na Lei de Goodhart?

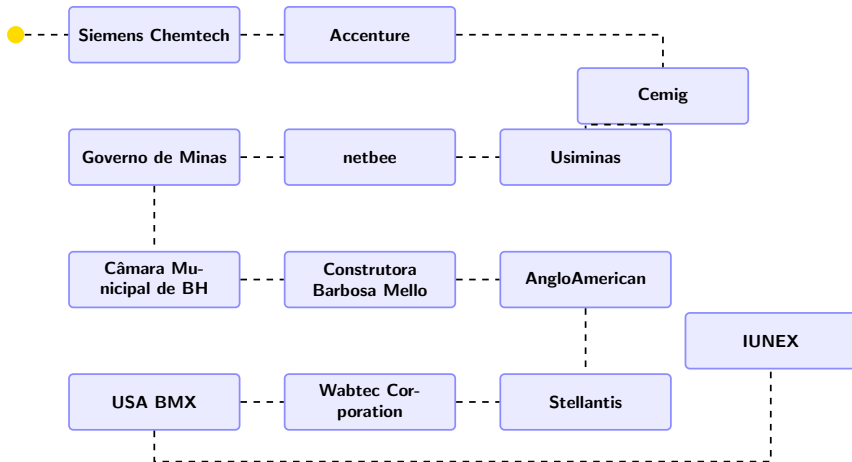
Guarde essa pergunta: vamos revisitá-la ao longo do semestre, quando desenharmos **experimentos controlados** para avaliar o real impacto de ferramentas de IA no desenvolvimento de software.

Danilo de Quadros Maia Filho

- Graduado em Engenharia Elétrica - UFMG
- LinkedIn:
<https://www.linkedin.com/in/danilo-maia-5299b925/>
- Executive MBA Gerenciamento de Projetos - FGV
- Mestre em Otimização - UFMG
- Projeto de Doutorado em andamento
- 17 anos de caminhada profissional
- Empresas: Stellantis, Anglo American, Wabtec Corporation, USABMX, Siemens, Accenture, Cemig (Projeto de Pesquisa)

Sobre o Professor

Projetos e experiências:



Dia Lourdes	Dia Coreu	Atividade
05/08/2026	04/08/26	Apresentação do curso
07/08/2026	06/08/26	Nivelamento
12/08/26	11/08/26	Nivelamento
14/08/26	13/08/26	Escopo de Medição
19/08/26	18/08/26	Escopo de Medição
21/08/26	20/08/26	Conceitos Básicos de Medição
26/08/26	25/08/26	Conceitos Básicos de Experimentação
28/08/26	27/08/26	Etapas de Experimentação e Relatórios
02/09/26	01/09/26	Entidades, Atributos e Objetivos de Medição
04/09/26	03/09/26	Entidades, Atributos e Objetivos de Medição
09/09/26	08/09/26	Métricas de Produto
11/09/26	10/09/26	Métricas de Produto
16/09/26	15/09/26	Métricas de Processo
18/09/26	17/09/26	Métricas de Projeto
23/09/26	22/09/26	Prova 1
25/09/26	24/09/26	Revisão de Prova
30/09/26	29/09/26	Pontos de Função
02/10/26	01/10/26	Pontos de Função
07/10/26	06/10/26	Estatística: Medidas de Tendência Central e Dispersão
09/10/26	08/10/26	Estatística: Variáveis Aleatórias Discretas e Contínuas
14/10/26	13/10/26	Estatística: Função de Densidade de Prob., Função Acumulada (Feriado)
16/10/26	15/10/26	Estatística: Distribuição Normal, Testes de Hipótese, Erros Tipo I e II
21/10/26	20/10/26	Enunciado do Trabalho Final
23/10/26	22/10/26	Prova 2

Figura 5: Cronograma - parte 1

28/10/26	27/10/26	Intrd. à Experimentação, Métodos e Estratégias Empíricas
30/10/26	29/10/26	Variáveis em Experimentos
04/11/26	03/11/26	Etapas: Escopo e Planejamento
06/11/26	05/11/26	Etapas: Operação
11/11/26	10/11/26	Etapas: Análise e Interpretação
13/11/26	12/11/26	Data Analysis - Data collection and Data Validation
18/11/26	17/11/26	Tomada de Decisão em Experimentos
20/11/26	19/11/26	Relatórios e Documentação Técnica
25/11/26	24/11/26	Prova 3
27/11/26	26/11/26	Apresentação do Trabalho Final
02/12/26	01/12/26	Apresentação do Trabalho Final
04/12/26	03/12/26	Revisão de Prova
09/12/26	08/12/26	Revisão de Matéria
11/12/26	10/12/26	Reavaliação
16/12/26	15/12/26	Revisão de Prova
21/12/26	17/12/26	

Figura 6: Cronograma - parte 2

- **Recepção dos Calouros do ICEI (*Campus* Coração Eucarístico): A SER DEFINIDA PELO INSTITUTO**
- **Recepção dos Calouros do *Campus* (*Campus* Lourdes): A SER DEFINIDA PELA PRO-REITORIA**
- **Aula Inaugural do ICEI: A SER DEFINIDA PELO INSTITUTO**
- **We Make Software - Mostra de TIs e TCCs:** 15/09/2026, 19:00 - 22:00 (*Campus* Lourdes) | 16/09/2026, 19:00 - 22:00 (*Campus* Coração Eucarístico)
- **Acolhida Integrada dos Calouros (*Campus* Coração Eucarístico): A SER DEFINIDA PELA SECOM**
- **Avaliação de Desempenho Acadêmico (ADA):** 16/11/2026, 08:50 - 22:30 (*Campus* Coração Eucarístico) | 19/11/2026, 08:50 - 22:30 (*Campus* Lourdes)

Figura 7: Datas do Semestre

- 1 Avaliação 1 - 25pts
- 2 Avaliação 2 - 25pts
- 3 Avaliação 3 - 25pts
- 4 Trabalho Final - 5pts
- 5 Participações e tarefas em sala de aula - 15pts
- 6 ADAS - 5pts

* O cronograma e a distribuição de pontos estão sujeitos a alterações

** Dentre os critério para concorrer aos pontos de **participações e tarefas**: cumprimento das tarefas solicitadas, postura em sala de aula, assistir às apresentações dos colegas.

A cada aula, 2 alunos da turma irão apresentar um artigo relacionado ao tema da última aula.

Critérios:

- Artigo de até 10 anos
- 5 minutos por aluno
- Apresentação oral ou utilizando quadro branco
- O datashow não será utilizado
- Apresentação individual
- Data sorteio ou a ser definida nos primeiros dias de aula
- Dia da apresentação não será transferível. O aluno que não apresentar terá nota de Participação descontada
- O professor fará um sorteio entre os demais alunos para realizar uma pergunta a quem apresentou o artigo.

Regras e Combinados

- ❶ Faltas não serão abonadas
- ❷ Não haverá arredondamento
- ❸ Respeitar o silêncio em sala de aula (participação)
- ❹ As chamadas poderão ser realizadas apenas uma vez em horário que pode variar OU 2x contando como 1 presença cada
- ❺ As chamadas não serão alteradas nem antes nem depois de realizadas
- ❻ O aluno tem de atingir 75% de frequência às aulas, como um dos critérios de aprovação nas disciplinas
- ❼ Os demais 25% são reservados para eventualidades, como, por exemplo, faltar por motivo de saúde
- ❽ BOM SENSO: Quando a chamada for realizada durante a aula ou ao final da aula, não serão aceitas respostas de alunos que estiveram ausentes durante a aula e retornaram apenas para responder a chamada

1 Introdução ao Curso

- Sobre a Disciplina
- Medição
- Experimentação
- Cuidado com as Métricas
- Métricas Modernas em Engenharia de Software
- Medição na Era da IA Generativa
- Sobre o Professor
- Planejamento da Disciplina
- Regras e Combinados

2 Primeiras Definições

Definição de Medição

Medição é o processo pelo qual números e símbolos são atribuídos a entidades do mundo real de forma a descrevê-las de acordo com regras claramente definidas.

Palavras importantes: Entidades, Atributos, números/símbolos, unidades, qualidades abstratas (alto/baixo, bom/ruim, rápido/lento, normal/exceção)

Definição alternativa de Medição

Medição é uma redução de incerteza quantitativamente expressa baseada em uma ou mais observações.

Palavras importantes: Observações, incerteza, quantitativo, probabilidade, expressar, redução.

Diferença v.s. Medição e Cálculo: *Measurement is a direct quantification, as in measuring the height of a tree or the weight of a shipment of bricks. Calculation is indirect, where we take measurements and combine them into a quantified item that reflects some attribute whose value we are trying to understand.*

- ❶ FENTON, Norman; BIEMAN, James. Software metrics: a rigorous and practical approach. CRC press, 2014.
- ❷ WOHLIN, Claes et al. Experimentation in software engineering. Berlin: Springer, 2012.
- ❸ JURISTO, Natalia; MORENO, Ana M. Basics of software engineering experimentation. Springer Science & Business Media, 2013.
- ❹ EMPIRICAL SOFTWARE ENGINEERING: an international journal. Boston: Kluwer Academic Publishers, 1996-. j20. ISSN 1573-7616. Disponível em: <https://link.springer.com/journal/10664>. Acesso em: 22 jul. 2024. (Periódico On-line).
- ❺ PRESSMAN, Roger S.; MAXIM, Bruce R.. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021. E-book. ISBN 9786558040118. (Livro Eletrônico).
- ❻ WOHLIN, Claes et al. Experimentation in software engineering. New York: Springer, c2012. xxiii, 236 p. ISBN 9783642432262. (Disponível no Acervo).

Bibliografia Complementar

- 1 JURISTO, Natalia. Basics of Software Engineering Experimentation. 1st ed. 2001. XXII, 396 p ISBN 9781475733044. (Livro Eletrônico).
- 2 LAIRD, Linda M.; BRENNAN, M. Carol,. Software measurement and estimation: a practical approach. Hoboken, N.J.: Wiley-Interscience, 2006. 1 online resource (xvi, 257 ISBN 0471792535 (electronic bk.) Disponível em :
[jhttps://ieeexplore.ieee.org/book/5988896](https://ieeexplore.ieee.org/book/5988896). Acesso em: 22 jul. 2024. (Livro Eletrônico).
- 3 SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson Education do Brasil, c2019. xii, 756 p. ISBN 9788543024974. (Disponível no Acervo).
- 4 DEVORE, Jay L. Probabilidade e estatística para engenharia e ciências. 3. São Paulo Cengage Learning 2018 1 recurso online ISBN 9788522128044. (Livro Eletrônico).
- 5 IEEE TRANSACTIONS ON SOFTWARE ENGINEERING. New York: IEEE Computer Society ,1975-. Mensal,. ISSN 0098-5589. Disponível em:
[https://ieeexplore.ieee.org/xpl/RecentIssue.jsp? punumber=32](https://ieeexplore.ieee.org/xpl/RecentIssue.jsp?punumber=32). Acesso em: 22 jul. 2024. (Periódico On-line).
- 6 JONES, Capers. Applied Software Measurement, 3rd edition. 2008. 1 online resource (696 pages). (Livro Eletrônico).
- 7 FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018. ISBN 9781942788331.
- 8 FORSGREN, Nicole et al. The SPACE of Developer Productivity: there's more to it than you think. ACM Queue, v. 19, n. 1, 2021.
- 9 STRATHERN, Marilyn. "Improving ratings": audit in the British University system. European Review, v. 5, n. 3, p. 305–321, 1997.